DigitalTwin for MyPalletizer 260 M5 Unity Version



Inhaltsverzeichnis

1. Abstract . . . 2
2. Introduction and Goals . . . 2
2.1. Functional Requirements . . . 3
2.2. Quality Goals . . . 3
2.3. Stakeholders . . . 4
3. Architecture Constraints . . . 4
3.1. Technische Randbedingungen . . . 4
3.2. Organisatorische Randbedingungen . . . 5
3.3. Konventionen und Standards . . . 5
3.4. Bewusste Grenzen und Annahmen . . . 5
4. Context and Scope . . . 5
4.1. Business Context . . . 5
4.2. Technical Context . . . 6
5. Solution Strategy . . . 7
5.1. Zuordnung der Qualitätsziele zu Lösungsansätzen . . . 7
5.2. Fundamentale Architekturentscheidungen . . . 8
6. Building Block View . . . 8
6.1. Whitebox Overall System (Level 1) . . . 9
6.2. Level 2: Subsysteme . . . 9
6.3. Level 3: Detailbausteine . . . 12
7. Runtime View . . . 13
7.1. Szenario 1: Asynchrone Gelenkbewegung (`send_angles`) . . . 14

7.2. Szenario 2: Synchrone Gelenkbewegung (sync_move_joints) . . . 15
7.3. Szenario 3: Endeffektor-Interaktion (Vakuumpumpe einschalten) . . . 16
8. Deployment View . . . 17
8.1. Infrastruktur- und Verteilungsdiagramm . . . 17
8.2. Bereitstellungsvarianten . . . 17
8.3. Softwarepaketierung . . . 18
9. Cross-cutting Concepts . . . 18
9.1. Simulations- und Kinematikkonzept . . . 18
9.2. Szenen- und Objektlebenszyklus . . . 18
9.3. Schutz- und Validierungskonzept . . . 19
9.4. Test- und Qualitätssicherungskonzept . . . 19
10. Architecture Decisions . . . 19
10.1. ADR-1: Einheitliche Python-API für Simulation und Hardware . . . 20
10.2. ADR-2: Kapselung der Herstellerbibliothek pymycobot . . . 20
10.3. ADR-3: Hybride Betriebsmodi (virtual, real, both) . . . 20
10.4. ADR-4: Trennung der Kommunikationskanäle (UDP Ports 5005 und 5006) . . . 21
10.5. ADR-5: Bereitstellung vordefinierter Szenarien über Presets . . . 21
10.6. ADR-6: Einsatz von ArticulationBody für die Gelenkphysik in Unity . . . 22
10.7. ADR-7: Pragmatische Nutzung herstellerseitiger CAD-Modelle mit Kompensation . . . 22
10.8. ADR-8: Verzicht auf optische Kamerasimulation zugunsten der Kernkinematik . . . 23
11. Quality Requirements . . . 23
11.1. Quality Tree . . . 23
11.2. Quality Scenarios . . . 24
11.3. Architektonische Zielkonflikte (Trade-offs) . . . 25
12. Risks and Technical Debts . . . 26
12.1. Architektur- und Technologierisiken . . . 26
12.2. Bekannte Einschränkungen und technische Schulden . . . 26

1. Abstract

Dieses Dokument beschreibt die Software- und Systemarchitektur des digitalen Zwillings für den Roboterarm MyPalletizer 260 M5 nach dem arc42-Standard. Das System ermöglicht die Entwicklung, den Test und die hybride Steuerung von Simulation (Unity) und physischer Roboterhardware über eine einheitliche, benutzerfreundliche Python-Schnittstelle. Es wurde im Rahmen des Moduls "**AI Robotics Challenge**" (AIRCH) an der Hochschule Luzern (HSLU) entwickelt.

2. Introduction and Goals

Ziel ist die Bereitstellung eines digitalen Zwillings für den 4-Achs-Palettierroboter MyPalletizer 260 M5. Die 3D-Simulationsumgebung ermöglicht das gefahrlose Entwickeln, Testen und Optimieren von Roboterprogrammen und Manipulationsszenarien in einer virtuellen Umgebung, bevor der

Code auf der physischen Hardware ausgeführt wird. Dadurch werden Kollisionen und mechanische Beschädigungen an den realen Servomotoren verhindert.

Das System löst den Engpass der beschränkten Hardwareverfügbarkeit im Lehrbetrieb, indem es Studierenden das ortsunabhängige Programmieren ermöglicht. Als Open Source Projekt steht die Lösung zudem externen Forschenden und Entwicklern zur Verfügung.

2.1. Functional Requirements

Anforderung	Beschreibung
3D-Visualisierung	Darstellung der Robotergeometrie, Gelenkbewegungen und Umgebungsobjekte in einer echtzeitfähigen 3D-Simulation (Unity).
Einheitliche Programmierschnittstelle	Bereitstellung einer Python-API, die identische Steuerbefehle für Simulation und physische Hardware akzeptiert, sodass Steuerungsskripte ohne Codeanpassung übertragbar sind.
Werkzeugunterstützung	Visuelle und funktionale Nachbildung der Endeffektoren: Parallelgreifer (**Gripper**) und Vakuumpumpe (**Suction Pump**).
Dynamische Szenenverwaltung	Erzeugen, Manipulieren und Löschen geometrischer Objekte (Würfel, Kugeln, Zylinder) zur Laufzeit sowie Laden vordefinierter Übungsszenarien (Presets).

2.2. Quality Goals

Die Architektur priorisiert die folgenden Qualitätsziele:

Rang	Qualitätsmerkmal	Szenario
1	**Austauschbarkeit**	Ein Steuerungsskript soll wahlweise die Simulation, den realen Roboter oder beide Systeme ansteuern können.
2	**Zeitverhalten & Latenz**	Kontinuierliche Gelenkbewegungen sollen flüssig und ohne sichtbare Verzögerung in der 3D-Szene visualisiert werden.
3	**Zuverlässigkeit & Fehlertoleranz**	Ungültige Parameter oder temporärer Paketverlust dürfen weder das Programm abbrechen noch zu unkontrollierten Bewegungen führen.
4	**Wartbarkeit & Modularität**	Neue Werkzeuge oder Simulationsszenarien sollen ergänzt werden können, ohne die Kernkinematik zu verändern.

Rang	Qualitätsmerkmal	Szenario
5	**Portabilität**	Das Python-Paket und die Unity-Simulation müssen unter Windows, Linux und macOS lauffähig sein.

2.3. Stakeholders

Rolle	Person / Gruppe	Hauptinteresse und Erwartung
Studierende	Endnutzer an der HSLU	Intuitive Python-API, verständliche Beispiele, gefahrloses Testen ohne Hardwarerisiko.
Dozierende	Prof. Dr. Jana Koehler (HSLU)	Zuverlässiges Werkzeug für Vorlesung und Projekte, saubere arc42-Dokumentation, reproduzierbare Übungsaufgaben.
Entwickler	Colin Felber (HSLU)	Modulare Architektur, hohe Wartbarkeit und erfolgreicher Modulabschluss.
Open-Source-Community	Externe Robotik-Entwickler	Einfache Installation und Bedienbarkeit.

3. Architecture Constraints

Die folgenden Vorgaben bilden den Rahmen für die Architektur des digitalen Zwillings. Wo die Implementierung eine Grenze setzt, ist diese ausdrücklich genannt.

3.1. Technische Randbedingungen

Randbedingung	Beschreibung und Auswirkung
Hardware-Plattform	Das System ist an den physischen 4-Achs-Roboter **MyPalletizer 260 M5** von Elephant Robotics gebunden. Dessen mechanische Achsgrenzen (`J1: -160..160°`, `J2: 0..90°`, `J3: -92..60°`, `J4: -360..360°`), Drehzahlen und Endeffektoren geben die Simulationsparameter vor.
Programmiersprache Python	Die Steuerungs- und Benutzerschnittstelle wird mit Python `>= 3.10` entwickelt. Diese Version ist in `python/pyproject.toml` festgelegt.
Simulations-Engine Unity	Als 3D-Engine ist Unity (C#) festgelegt.
Plattformunabhängigkeit	Sowohl das Python-Paket als auch die kompilierte Unity-Simulation müssen ohne systemspezifische Anpassungen auf Windows, Linux und macOS lauffähig sein.

3.2. Organisatorische Randbedingungen

Randbedingung	Beschreibung und Auswirkung
Hochschul- und Semesterrahmen	Das Projekt wurde als Einzelprojektarbeit im Modul AI-Robotics an der Hochschule Luzern innerhalb eines Semesters umgesetzt.

3.3. Konventionen und Standards

- Die öffentliche API liegt in Python und wird über `Robot` und `World` verwendet.
- Nachrichten zwischen Python und Unity sind UTF-8-kodierte JSON-Datagramme über UDP.
- Die vier Gelenke werden durchgehend als `j1` bis `j4` bezeichnet.
- Architekturentscheidungen werden in Kapitel 9 als ADRs festgehalten.

3.4. Bewusste Grenzen und Annahmen

Zur Wahrung des Projektumfangs wurden bewusste Systemgrenzen definiert:

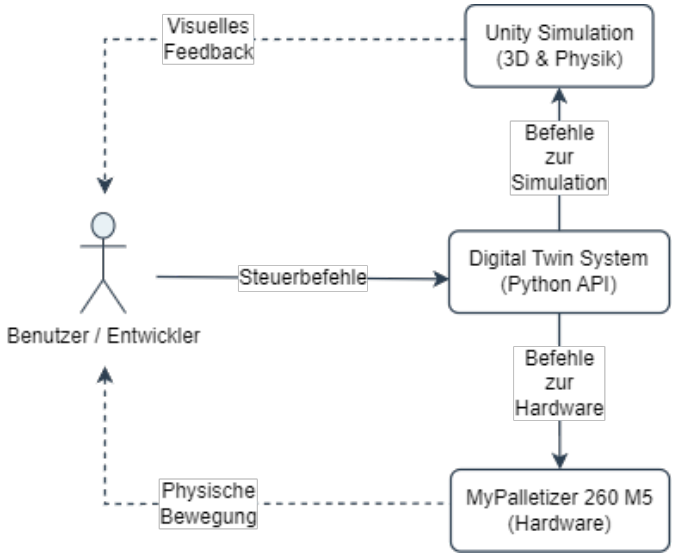
- **Open-Loop-Simulation:** Python sendet Befehle an Unity ohne Rückmeldung oder simulierte Sensorik (Details siehe ADR-1 und Kapitel 11).
- **Vereinfachte Interaktionsphysik:** Werkzeuge koppeln Objekte kinematisch (`AttachTo`); auf komplexe Reibungskräfte und virtuelle Kameras wird verzichtet (siehe ADR-7 und ADR-8).
- **Feste Portzuordnung:** Die Kommunikation nutzt standardmässig die Ports `5005` (Roboter) und `5006` (Welt). Portänderungen müssen in Python und im Unity-Inspector synchron gepflegt werden (siehe Kapitel 3 und 11).

4. Context and Scope

Dieses Kapitel grenzt das System gegenüber externen Akteuren und angebundenen Nachbarsystemen ab. Es definiert die fachlichen Interaktionsbeziehungen (Business Context) sowie die technischen Kommunikationswege und Protokolle (Technical Context).

4.1. Business Context

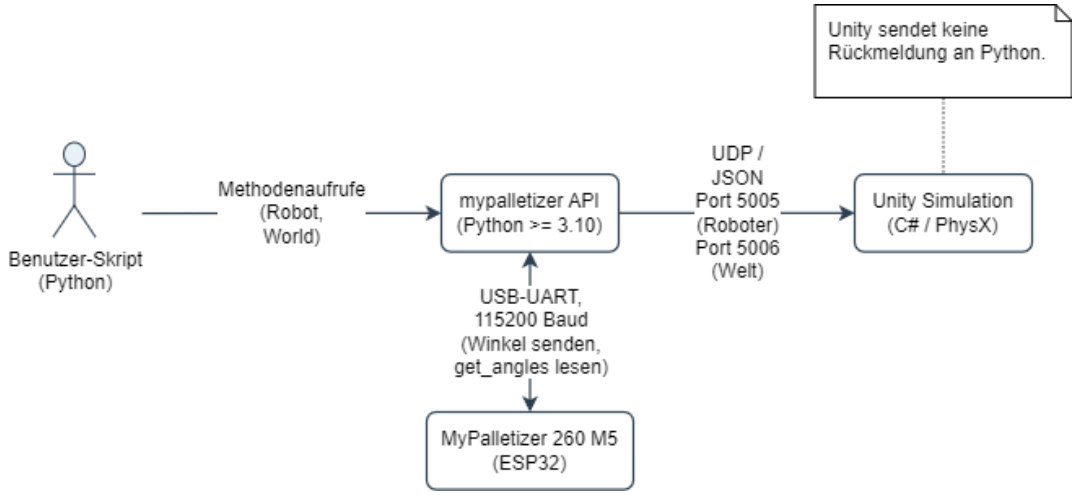
Der fachliche Kontext beschreibt die Systemgrenzen aus der Anwenderperspektive. Der Benutzer steuert das Gesamtsystem ausschliesslich über die bereitgestellte Python-Bibliothek. Das System fungiert als Vermittler, validiert Parameter und leitet Aktionen je nach gewähltem Modus an die 3D-Simulation und/oder die physische Roboterhardware weiter.



Akteur / Nachbarsystem	Fachliche Rolle	Fachliche Ein- und Ausgaben
Benutzer / Entwickler	Schreibt Steuerungsskripte in Python; plant Bewegungsabläufe und Pick-and-Place-Szenarien.	**Eingabe:** Steuerbefehle (Winkel, Geschwindigkeiten, Werkzeugaktionen, Szenenobjekte). **Ausgabe:** Visuelles Feedback in der Simulation und/oder physische Bewegung der Hardware.
Digital Twin System	Kapselt die interne Ablaufsteuerung, prüft Grenzwerte und steuert die Zielsysteme modusspezifisch an.	**Eingabe:** API-Methodenaufrufe des Benutzers. **Ausgabe:** JSON-Datagramme an Unity; serielle Steuerpakete an die Hardware.
Unity Simulation	3D-Echtzeit-Visualisierung von Kinematik, Werkzeugen und dynamischen Simulationskörpern.	**Eingabe:** UDP-Steuerkommandos. **Ausgabe:** Grafische Darstellung der Bewegungen und Greifvorgänge. Unity sendet derzeit keine Zustandsquittung an Python.
MyPalletizer 260 M5	Physischer Roboterarm zur realen Ausführung von Bewegungsabläufen.	**Eingabe:** Serielle Befehlspakete. **Ausgabe:** Physische Gelenkbewegungen und Rückmeldung des aktuellen Hardwarezustands.

4.2. Technical Context

Der technische Kontext definiert die konkreten Schnittstellen, Medien und Übertragungsprotokolle zwischen dem Python-Steuerungssystem und den Nachbarsystemen.



Schnittstelle	Übertragungsmedium	Protokoll / Format	Technische Spezifikation
Benutzerskript → Python API	In-Process Funktionsaufruf	Python API	Direkte Methodenaufrufe auf den Klassen `Robot` und `World` mit nativer Parameterübergabe und Typisierung.
Python API → Unity (Roboterkanal)	UDP Socket (Localhost)	UTF-8 JSON, Port `5005`	Übertragung zeitkritischer Roboterkommandos (`move_joints`, `sync_move_joints`, `led`, `set_end_effector`, `set_gripper_state`, `pump`).
Python API → Unity (Weltkanal)	UDP Socket (Localhost)	UTF-8 JSON, Port `5006`	Übertragung von Szenenverwaltungsbefehlen (`spawn`, `delete`, `clear`) inklusive 3D-Transformationsvektoren und Farbdaten.
Python API <→ MyPalletizer Hardware	USB-UART (Serielle Schnittstelle)	`115200` Baud (via `pymycobot`)	Bidirektionale Kommunikation: Senden von Zielwinkeln und Relaiszuständen; Auslesen von Sensor- und Gelenk-Telemetriedaten.

5. Solution Strategy

Die Lösungsstrategie fasst die grundlegenden architektonischen Entwurfsentscheidungen zusammen, mit denen die Qualitätsziele aus Kapitel 1 erreicht werden. Detaillierte Bausteine, übergreifende Mechanismen und begründete Architekturentscheidungen werden in den Folgekapiteln vertieft.

5.1. Zuordnung der Qualitätsziele zu Lösungsansätzen

Qualitätsziel	Architektonische Herausforderung	Gewählter Lösungsansatz	Details
1. Austauschbarkeit	Steuerungsskripte sollen identisch für Simulation und Hardware funktionieren.	Einheitliche Fassade (`Robot`), modusspezifisches Routing (`virtual`, `real`, `both`) im Controller verborgen.	Kap. 5, ADR-1, ADR-3

Qualitätsziel	Architektonische Herausforderung	Gewählter Lösungsansatz	Details
2. Zeitverhalten & Latenz	Flüssige 3D-Visualisierung ohne Blockieren des Benutzerskripts.	Verbindungsloses UDP für Bewegungsdaten; Entkopplung von Empfang und Rendering über interne Unity-Queues.	Kap. 8, ADR-4
3. Zuverlässigkeit	Schutz vor Hardwarebeschädigung und fehlerhaften Benutzereingaben.	Zweistufiges Schutzkonzept: Frühes Abfangen struktureller Fehler (Fail-Fast) und automatisches Begrenzen numerischer Werte (Clamping).	Kap. 8
4. Wartbarkeit & Modularität	Unabhängige Erweiterbarkeit von Robotersteuerung und Simulationsszene.	Strikte Schichtentrennung (Python vs. Unity) sowie getrennte Kommunikationskanäle (Port 5005 und 5006).	Kap. 3, Kap. 5, ADR-4
5. Portabilität	Plattformunabhängige Ausführung unter Windows, Linux und macOS.	Standard-Python ohne OS-spezifische Erweiterungen; plattformunabhängige Unity-Standalone-Builds.	Kap. 2, Kap. 7

5.2. Fundamentale Architekturentscheidungen

Die wesentlichen Entwurfsentscheidungen sind in Kapitel 9 als Architectural Decision Records (ADR) dokumentiert:

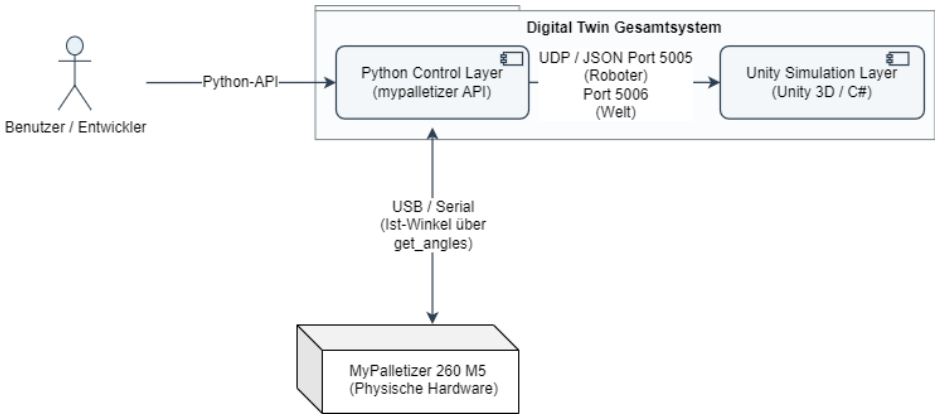
- **Einheitliche Fassade:** Kapselung von `pymycobot` und Netzwerk-Sockets hinter `Robot` und `World` (→ ADR-1, ADR-2).
- **Parallele Kommunikationskanäle:** Port 5005 für hochfrequente Roboterkommandos, Port 5006 für Szenenverwaltung (→ ADR-4).
- **Gelenkphysik mit ArticulationBody:** Stabile physikalische Mehrkörpersimulation ohne numerische Gelenkdehnung (→ ADR-6).
- **Kinematische Werkzeugkopplung:** Pragmatische Objektanbindung (`AttachTo`) statt aufwendiger Reibungssimulation (→ ADR-7).
- **Fokus auf Kernkinematik:** Verzicht auf rechenintensive virtuelle Bildverarbeitung in Unity (→ ADR-8).

6. Building Block View

Die Bausteinsicht zeigt die statische Struktur des Gesamtsystems auf drei Detaillierungsebenen: Gesamtsystem (Level 1), Subsysteme (Level 2) und Detailkomponenten (Level 3).

6.1. Whitebox Overall System (Level 1)

Das System besteht aus dem **Python Control Layer** (Steuerung, Validierung, API) und dem **Unity Simulation Layer** (3D-Visualisierung, Kinematik, Physik). Der physische **MyPalletizer 260 M5** ist als externes Zielsystem angebunden.

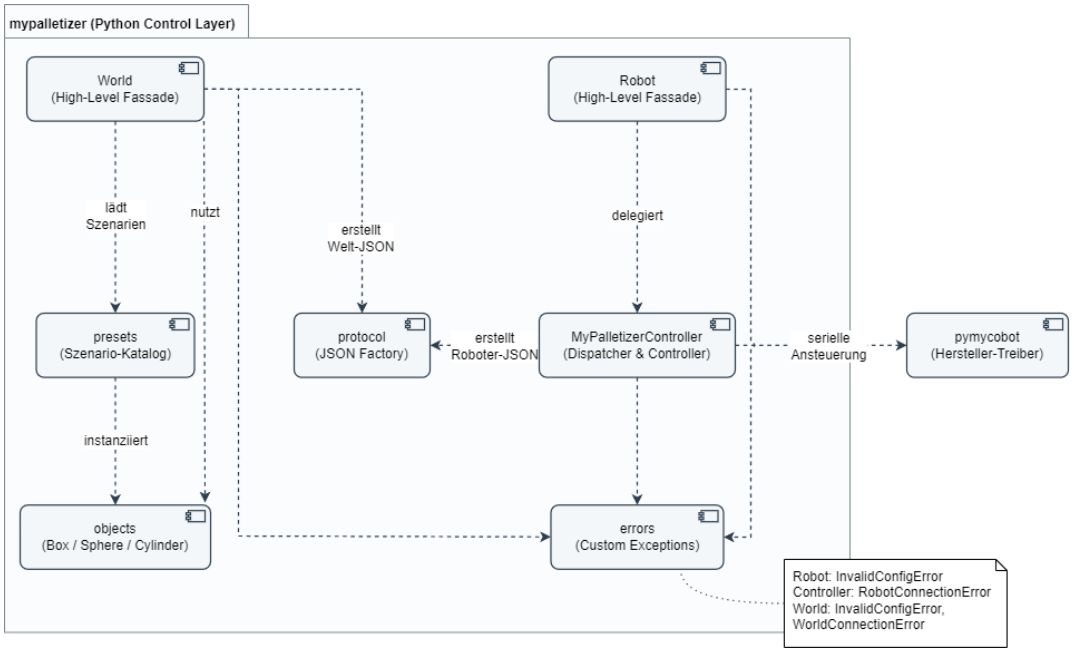


Baustein	Verantwortung und Schnittstellen
Python Control Layer	Öffentliche API für Benutzer; Validierung und Begrenzung von Steuerwerten; modusspezifische Weiterleitung an Simulation (UDP) und/oder Hardware (Serial).
Unity Simulation Layer	Empfang von UDP-Nachrichten; 3D-Visualisierung des Roboterarms und der Werkzeuge; physikalische Gelenkbewegung via `ArticulationBody`; Verwaltung virtueller Objekte.
MyPalletizer 260 M5	Physische Hardware; Ausführung von Bewegungsbefehlen und Werkzeugaktionen über interne Servomotoren und Relais.
Benutzer / Entwickler	Erstellt Steuerungsskripte unter ausschliesslicher Verwendung der Klassen `Robot` und `World`.

6.2. Level 2: Subsysteme

6.2.1. Whitebox Python Control Layer

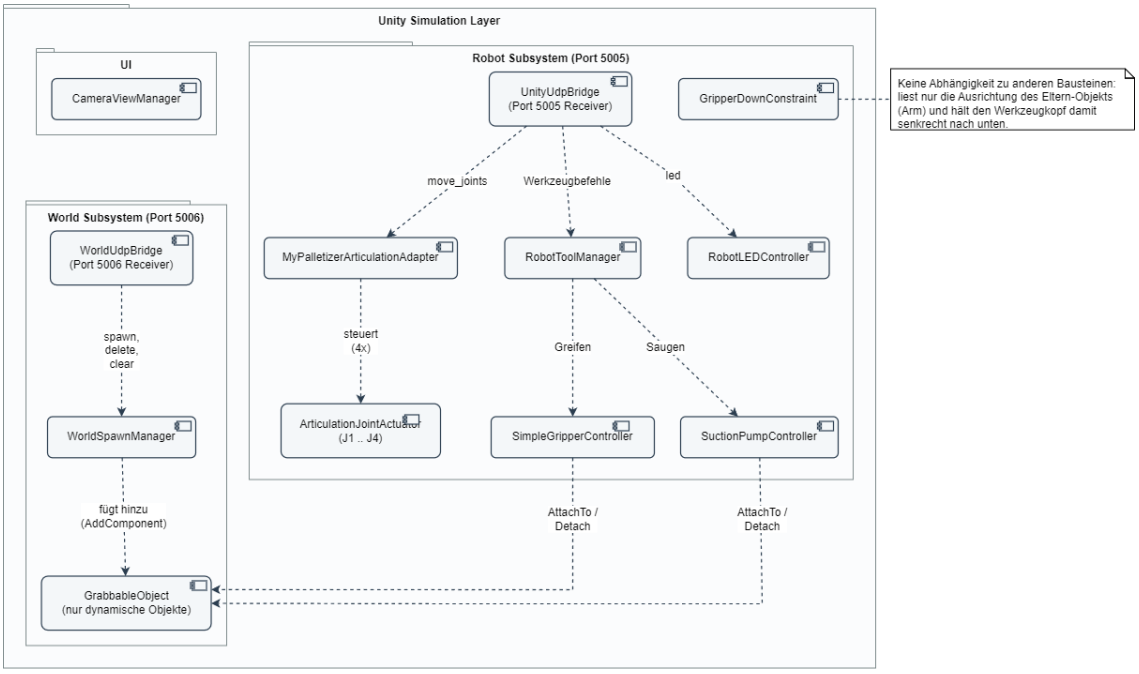
Der Python Control Layer gliedert sich in benutzernahe Fassadenklassen, den zentralen Controller sowie Hilfsmodule für Protokollerstellung, vordefinierte Presets, Datenklassen und Ausnahmen.



Baustein	Verantwortung und Aufgaben
`Robot`	High-Level-Fassade für die Robotersteuerung (`send_angles`, `sync_move_joints`, `set_color`, `set_end_effector`, `set_gripper_state`, `pump_on`, `pump_off`). Dazu kommen `send_angle` (ein einzelnes Gelenk), `get_angles`, `sleep` und `close`. `Robot` kann auch als `with`-Block verwendet werden, dann wird `close()` automatisch aufgerufen.
`MyPalletizerController`	Zentraler Dispatcher und Treiber. Führt Software-Clamping für Winkel, Geschwindigkeit und RGB durch; verwaltet den lokalen Zustandsspeicher `_sim_angles`; hält den UDP-Socket für Port `5005`; kapselt die `MyPalletizer260`-Hardwareinstanz.
`World`	High-Level-Fassade für die Umgebungsverwaltung (`spawn`, `delete`, `clear`, `load_preset`). Sendet über einen separaten UDP-Socket auf Port `5006`.
`protocol`	Zentrale Factory zur Konstruktion einheitlicher JSON-Nachrichten mit Versionsfeld (`"v": 1`) und Typkennung (`"type"`).
`presets`	Katalog der implementierten Szenarien `empty`, `three_blocks`, `Base1` und `sorting_task`.
`objects`	Datenklassen für 3D-Körper: `WorldObject` (Basisklasse) sowie `Box`, `Sphere` und `Cylinder`.
`errors`	Benutzerdefinierte Ausnahmen (`InvalidConfigError`, `RobotConnectionError`, `WorldConnectionError`).
`pymycobot`	Externe Herstellerbibliothek zur seriellen Kommunikation mit dem MyPalletizer 260 M5.

6.2.2. Whitebox Unity Simulation Layer

Die Simulationsarchitektur in Unity trennt strikt zwischen Roboter-Subsystem (Port `5005`) und Welt-Subsystem (Port `5006`).

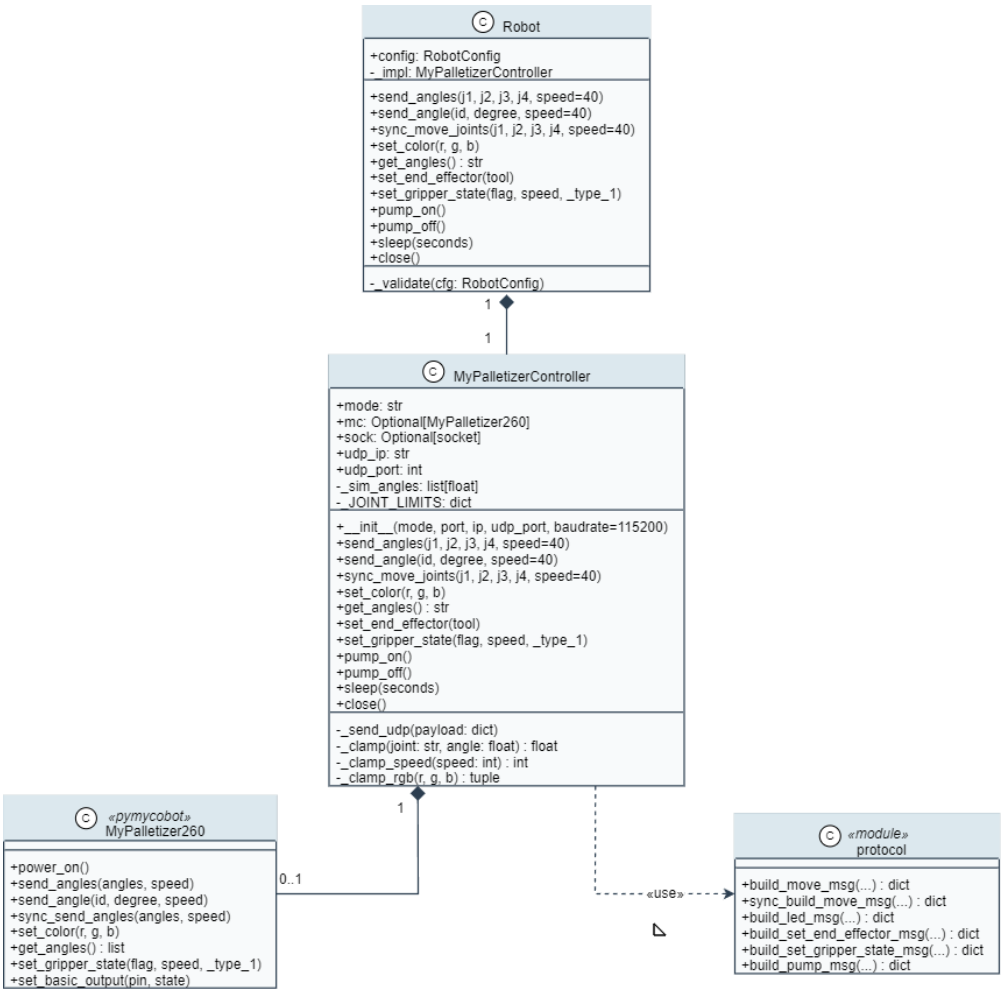


Baustein	Verantwortung und Aufgaben
`UnityUdpBridge`	Empfängt UDP-Datagramme auf Port `5005` in einem Hintergrund-Thread und reiht sie threadsicher in eine Queue ein. Der `CommandRunner` leitet die Befehle im Hauptthread weiter.
`MyPalletizerArticulationAdapter`	Koordiniert die vier Gelenkaktuatoren `j1` bis `j4`. Bietet `MoveJointsAsync` (asynchron) und `MoveJointsSync`.
`ArticulationJointActuator`	Steuert den `ArticulationBody` eines einzelnen Gelenks über dessen `xDrive` und fährt das Gelenk mit einer Coroutine in kleinen Schritten zum Zielwinkel (ein Schritt pro `FixedUpdate`, Geschwindigkeit abhängig vom `speed`-Wert).
`GripperDownConstraint`	Hält den Werkzeugkopf im `LateUpdate` vertikal nach unten ausgerichtet (Kompensation der Palettiermechanik).
`RobotToolManager`	Schaltet die Sichtbarkeit der Endeffektor-Modelle (`gripperTool` / `pumpTool`) und leitet Aktionen an den aktiven Werkzeugcontroller weiter.
`SimpleGripperController`	Führt mechanische Greifvorgänge aus; erfasst `GrabbableObject`-Objekte via `Physics.OverlapSphere` und bindet sie kinematisch an den Greifer.
`SuctionPumpController`	Steuert die Vakuumpumpe; koppelt das nächstgelegene `GrabbableObject` bei aktivem Unterdruck an den Saugkopf.
`RobotLEDController`	Steuert die RGB- und Emissionsdarstellung der Basis-LEDs des simulierten M5-Moduls.
`WorldUdpBridge`	Empfängt UDP-Datagramme auf Port `5006` und übergibt sie im `Update` -Frame an den `WorldSpawnManager`.
`WorldSpawnManager`	Instanziiert Prefabs (`boxPrefab`, `spherePrefab`, `cylinderPrefab`), konfiguriert `Rigidbody`, weist Farben zu und verwaltet aktive Objekte (`_spawnedObjects`). Nur dynamische Objekte erhalten zusätzlich `GrabbableObject`.

Baustein	Verantwortung und Aufgaben
`GrabbableObject`	Kennzeichnungskomponente für interaktionsfähige Objekte; verwaltet hierarchische Fixierung (`AttachTo`) und Freigabe (`Detach`).
`CameraViewManager`	Reine Bedienhilfe der Simulation: Wechselt per Dropdown zwischen vordefinierten Kameraperspektiven. Das ist keine Kamerasimulation im Sinne von ADR-8 und hat keine Verbindung zu Python.

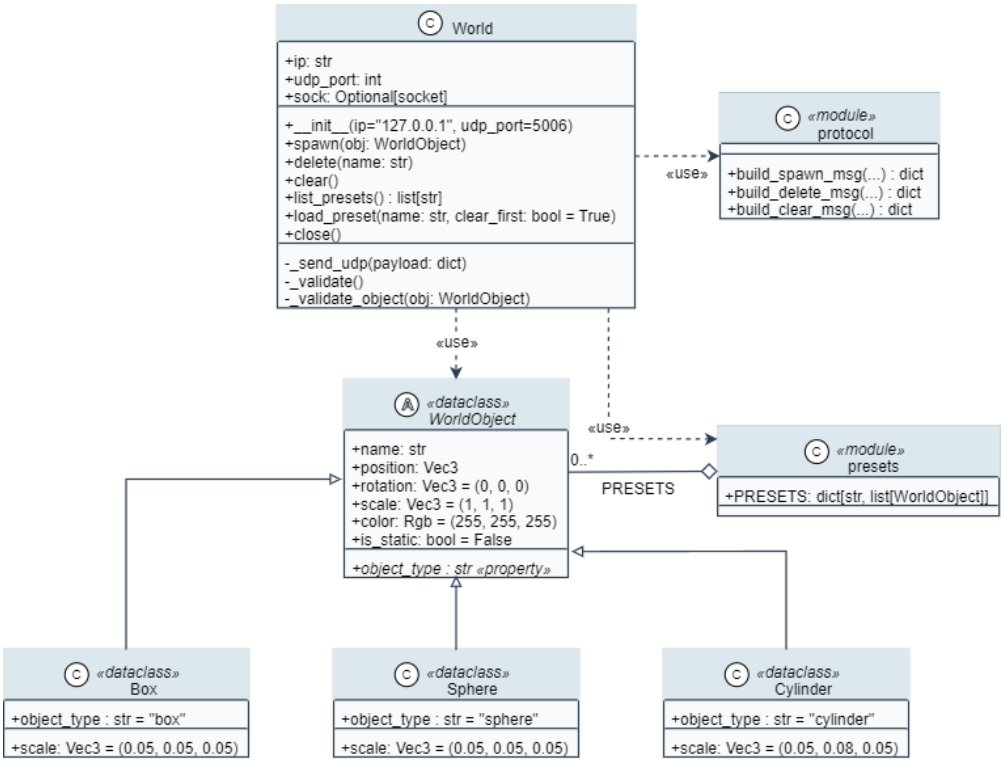
6.3. Level 3: Detailbausteine

6.3.1. `MyPalletizerController` (Klassendiagramm)



- **Gelenkwinkelbegrenzung (`_clamp`):** Schützt die Hardware durch hartcodierte Grenzwerte (`j1: -160..160°`, `j2: 0..90°`, `j3: -92..60°`, `j4: -360..360°`).
- **Telemetrie-Simulation (`get_angles`):** Gibt im Modus `virtual` die Werte aus `_sim_angles` und im Modus `real`/`both` die von der Hardware gelesenen Gelenkdaten zurück.
- **Einzelgelenk (`send_angle`):** Diese Methode schreibt nur an die Hardware. Im Modus `virtual` löst sie `NotImplementedError` aus, im Modus `both` bewegt sie nur den echten Roboter und nicht die Simulation (siehe Kapitel 11).
- **Werkzeugwahl (`set_end_effector`):** Die Auswahl `gripper` oder `pump` geht nur an Unity. Der echte Roboter hat kein Gegenstück dazu, dort wird das Werkzeug mechanisch montiert.

6.3.2. `World` und `WorldObject` (Klassendiagramm)



- **Validierung (`_validate_object`):** Prüft vor dem Senden, dass Objektnamen nicht leer sind, 3D-Vektoren 3 Elemente enthalten und Skalierungswerte strikt positiv (`> 0`) sind.

6.3.3. `protocol` (Nachrichtenformate)

Die folgende Tabelle zeigt beispielhaft einige Nachrichten zwischen dem Python Control Layer und dem Unity Simulation Layer:

Nachrichtentyp (`type`)	Port	Beispiel-Struktur (JSON)
`"move_joints"`	5005	`{"v":1,"type":"move_joints","j1":0.0,"j2":20.0,"j3":-30.0,"j4":0.0,"speed":40}`
`"sync_move_joints"`	5005	`{"v":1,"type":"sync_move_joints","j1":10.0,"j2":45.0,"j3":0.0,"j4":90.0,"speed":50}`
`"led"`	5005	`{"v":1,"type":"led","r":0,"g":255,"b":0}`
`"set_gripper_state"`	5005	`{"v":1,"type":"set_gripper_state","flag":1,"speed":50,"type_1":1}`
`"pump"`	5005	`{"v":1,"type":"pump","enabled":true}`
`"spawn"`	5006	`{"v":1,"type":"spawn","object_type":"box","name":"b1","position":{"x":0.1,"y":0.2,"z":0.3},"rotation":{"x":0,"y":0,"z":0},"scale":{"x":0.05,"y":0.05,"z":0.05},"color":{"r":255,"g":0,"b":0},"is_static":false}`

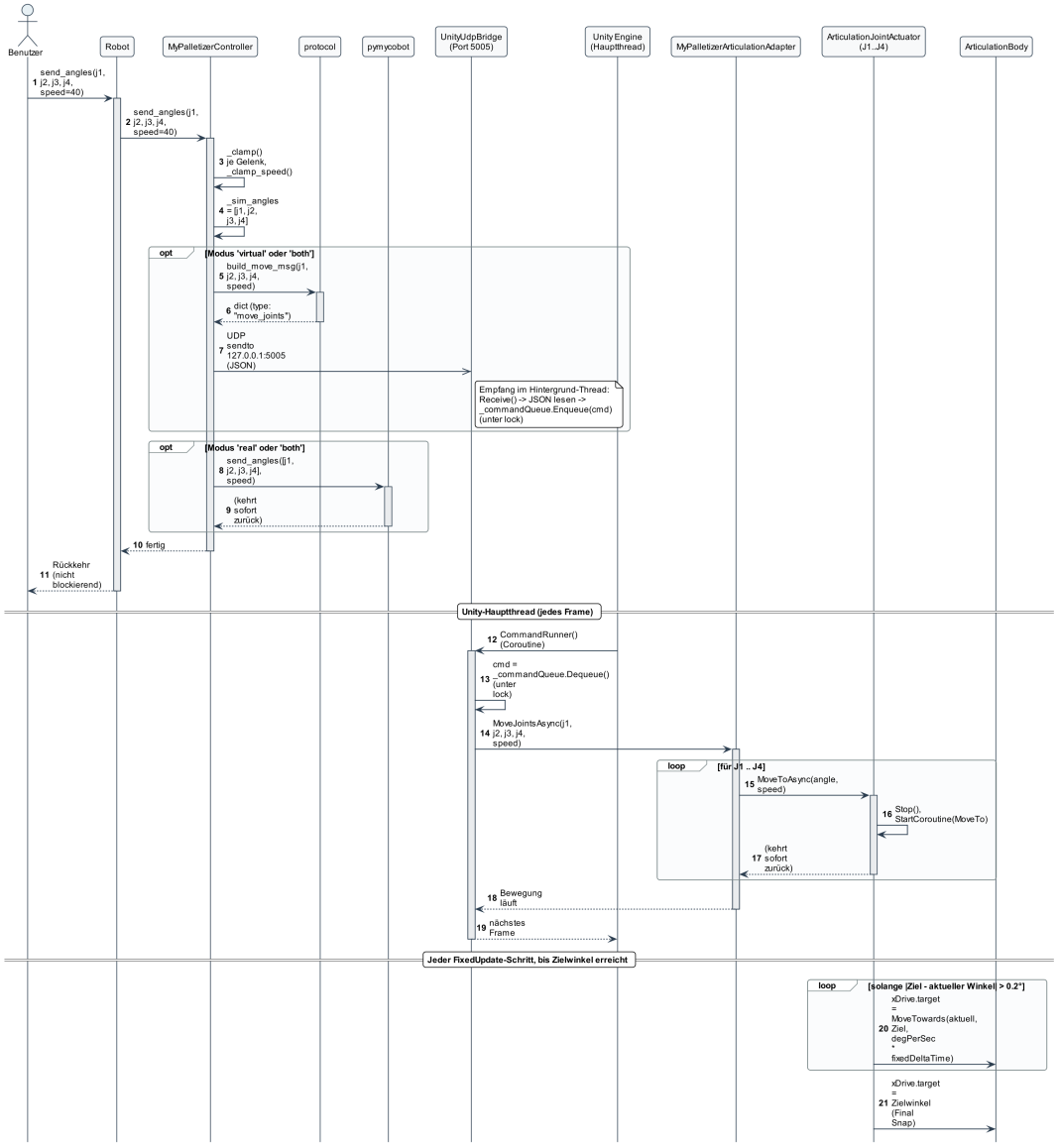
7. Runtime View

Die Laufzeitsicht veranschaulicht das dynamische Zusammenspiel der Bausteine anhand der architektonisch relevanten Kernabläufe: der asynchronen und synchronen Gelenkansteuerung

sowie der Werkzeug- und Objektmanipulation.

7.1. Szenario 1: Asynchrone Gelenkbewegung (`send_angles`)

In diesem Szenario sendet der Benutzer Gelenkwinkel im "Fire-and-Forget"-Modus. Der Aufruf kehrt sofort zum Aufrufer zurück, während die Bewegung parallel in Unity und auf der Hardware ausgeführt wird.

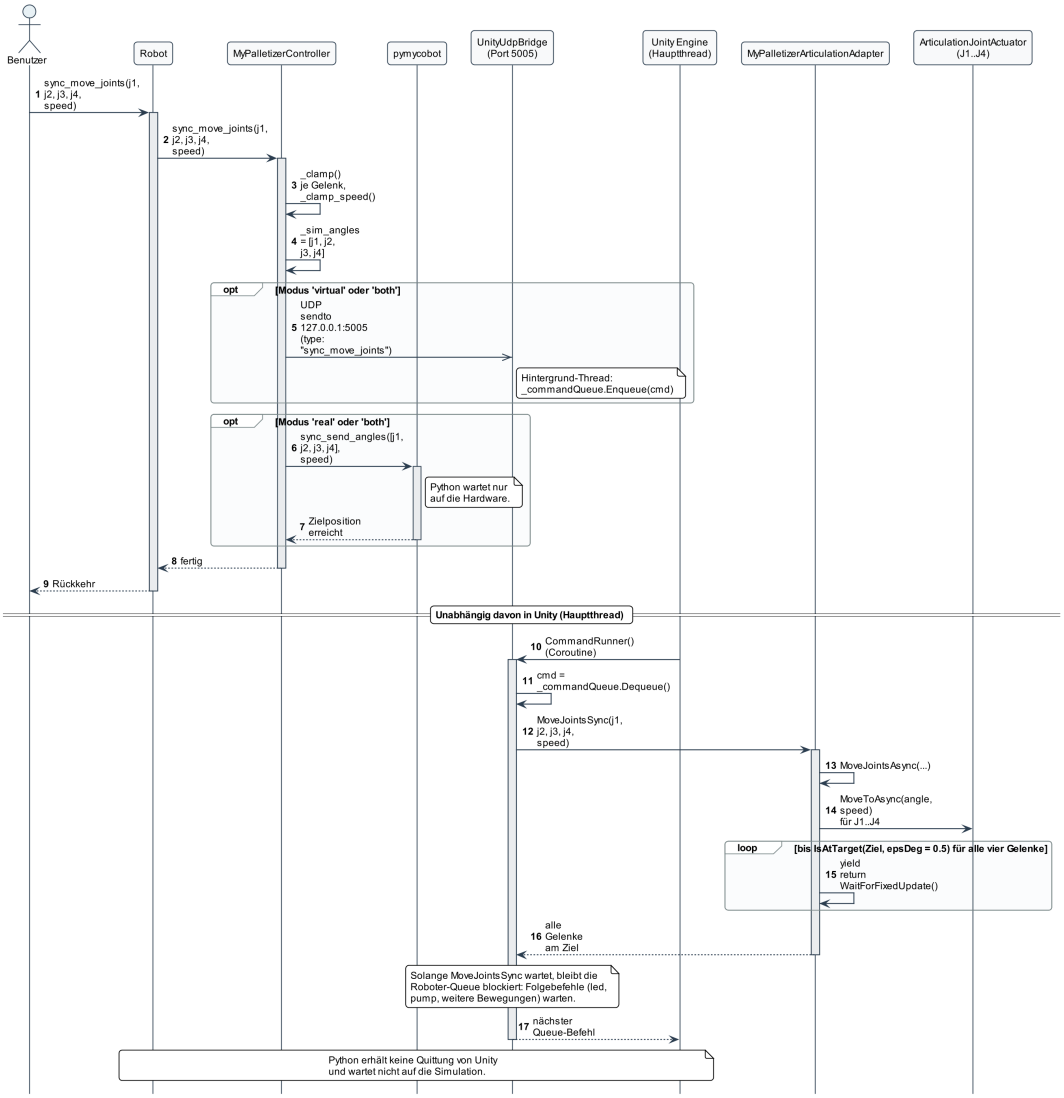


1. **API-Aufruf & Clamping:** Der Benutzer ruft `send_angles()` auf. Der Controller begrenzt Winkel und Geschwindigkeit auf hardwarekonforme Bereiche und aktualisiert den lokalen Zustandsspeicher `_sim_angles`.
2. **JSON-Erstellung & UDP-Versand:** `protocol.build_move_msg()` erzeugt ein Datagramm (`type: "move_joints"`), das über einen UDP-Socket an Port `5005` gesendet wird.
3. **Hardwareübertragung:** Parallel dazu wird `send_angles()` an die Herstellerbibliothek `pymycobot` übergeben.
4. **Abarbeitung in Unity:** `UnityUdpBridge` empfängt das Datagramm im Hintergrund-Thread und reiht es in die threadsichere `_commandQueue` ein. Der `CommandRunner` holt den Befehl im

Hauptthread ab und übergibt die Zielwinkel an `MoveJointsAsync`. Die Coroutine im `ArticulationJointActuator` führt die Gelenke pro `FixedUpdate`-Schritt gleichmässig nach.

7.2. Szenario 2: Synchrone Gelenkbewegung (`sync_move_joints`)

Dieses Szenario beschreibt das koordinierte Warten auf das Erreichen der Zielposition bei physischem Roboter und virtueller Simulation.

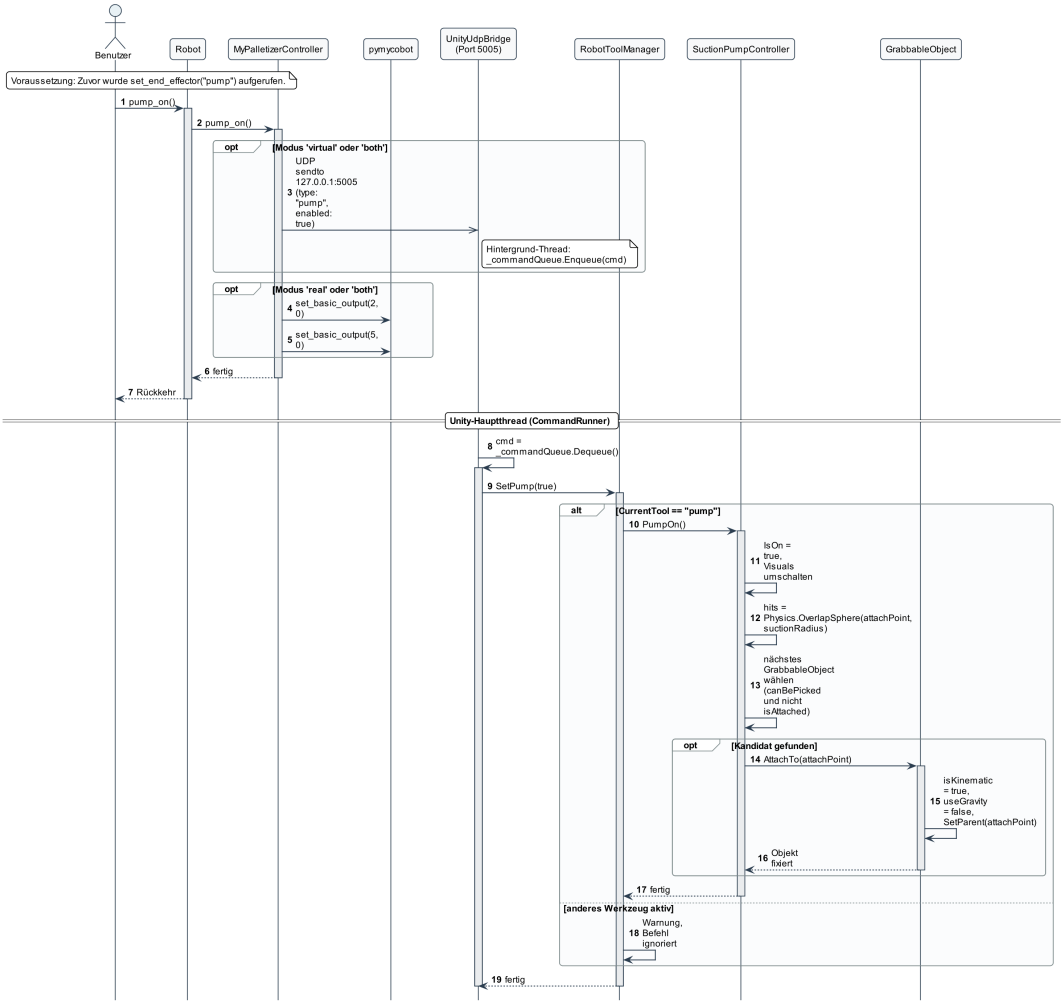


1. **API-Aufruf:** Der Benutzer ruft `sync_move_joints()` auf.
2. **Hardware-Synchronisation:** Der Controller ruft `sync_send_angles()` auf `pymycobot` auf. Diese Methode blockiert den Python-Thread, bis die physischen Servomotoren die Zielposition eingenommen haben.
3. **Simulation:** Die `UnityUdpBridge` empfängt den Befehl und übergibt ihn an `MoveJointsSync(…)`. Unity wartet in einer Coroutine, bis alle vier Gelenke innerhalb des Toleranzbereichs von `0.5°` liegen. Python erhält darüber keine Rückmeldung und wartet nicht auf Unity.
4. **Queue-Verhalten:** Während Unity auf den Abschluss von `MoveJointsSync` wartet, bleibt die interne Roboter-Befehlswarteschlange blockiert. Nachfolgende Roboterbefehle (z. B. LED oder Werkzeugaktionen) werden erst nach Erreichen der Zielpose abgearbeitet. Weltbefehle auf Port

`5006` bleiben davon unberührt.

7.3. Szenario 3: Endeffektor-Interaktion (Vakuumpumpe einschalten)

Dieses Szenario zeigt die Aktivierung eines Werkzeugs und das physikalische Greifen eines Objekts in der Simulationsumgebung.



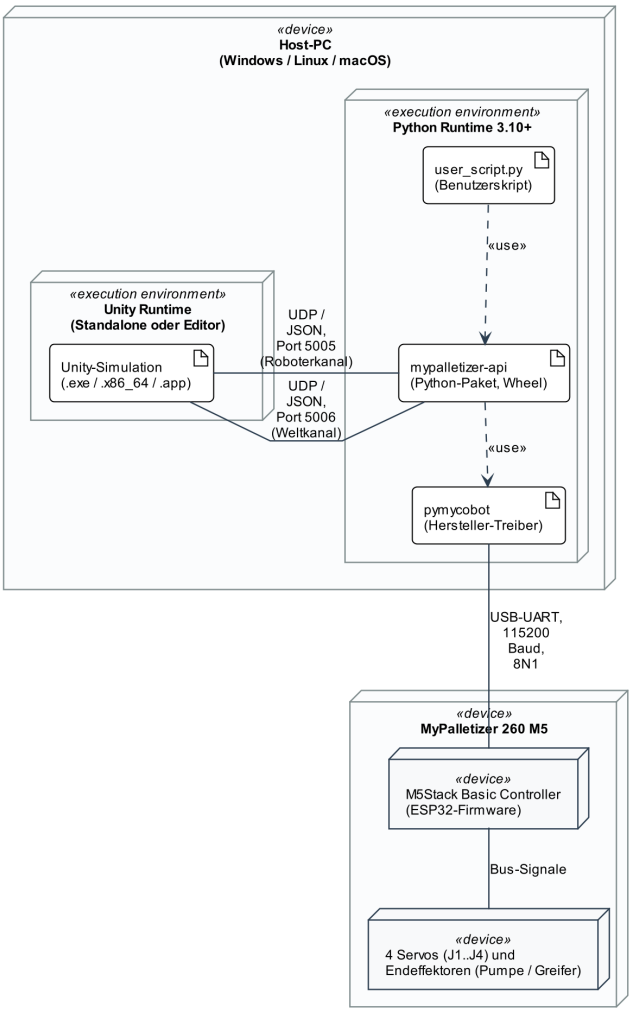
1. **API-Aufruf & Vorbedingung:** Der Benutzer ruft `robot.pump_on()` auf. Voraussetzung in Unity ist, dass zuvor `robot.set_end_effector("pump")` aufgerufen wurde; andernfalls ignoriert Unity den Pumpenbefehl mit einer Log-Warnung.
2. **Hardware-Aktivierung:** Der Controller schaltet die GPIO-Pins über `pymycobot.set_basic_output()` auf Low (`0`), wodurch die physischen Pumpenrelais anziehen.
3. **Simulations-Aktivierung:** Ein UDP-Paket (`type: "pump"`, `enabled: true`) wird an Port `5005` gesendet.
4. **Greiflogik in Unity:** Der `SuctionPumpController` schaltet die optische Rückmeldung ein und prüft mittels `Physics.OverlapSphere` den Bereich um den Saugkopf auf Objekte mit der Komponente `GrabbableObject`. Wird ein solches Objekt erfasst, wird es über `obj.AttachTo(attachPoint)` kinematisch an den Endeffektor gekoppelt.

8. Deployment View

Die Verteilungssicht beschreibt die physischen Hardwareknoten, die Software-Laufzeitumgebungen und die Netzwerkverbindungen des Gesamtsystems.

8.1. Infrastruktur- und Verteilungsdiagramm

Das System wird auf einem lokalen Arbeitsplatzrechner (**Host-PC**) ausgeführt. Optional ist der physische Roboter **MyPalletizer 260 M5** über eine serielle USB-Verbindung angeschlossen.



8.2. Bereitstellungsvarianten

- **Reine Simulation (`virtual`):** Python-Skript und Unity-Anwendung laufen auf demselben Rechner; kein Hardwareanschluss ist erforderlich. Unity muss laufen, sonst werden die UDP-Befehle nicht simuliert.
- **Hybrider Parallelbetrieb (`both`):** Python-Skript steuert zeitgleich die Unity-Simulation und den über USB verbundenen Roboter an. Dient dem direkten visuellen Soll-Ist-Vergleich.
- **Reiner Hardwarebetrieb (`real`):** Python-Skript steuert ausschliesslich den physischen Roboter an; Unity muss nicht gestartet werden.

8.3. Softwarepaketierung

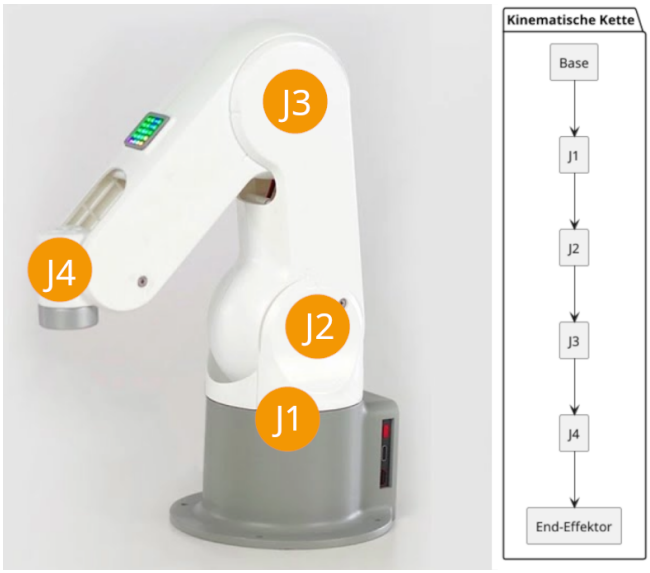
- **Python-Bibliothek:** Paketiert als Standard-Wheel via `pyproject.toml` (`pip install .`).
- **Unity-Simulation:** Bereitstellung als eigenständige Standalone-Binaries für Windows (`.exe`), Linux (`.x86_64`) und macOS (`.app`) über automatisierte GitHub Actions.

9. Cross-cutting Concepts

In diesem Kapitel werden die übergreifenden Entwurfsmuster, Architekturkonzepte und Implementierungsprinzipien beschrieben, die mehrere Bausteine des Systems prägen.

9.1. Simulations- und Kinematikkonzept

Die Kinematik des Roboters wird in Unity realitätsnah und stabil nachgebildet.



- **Mehrkörper-Kinematik mit `ArticulationBody`:** Die vier Drehachsen (`J1` bis `J4`) sind baumförmig angeordnet. Unitys `ArticulationBody` verhindert numerische Dehnungen und sorgt für ein physikalisch stabiles Verhalten der verbundenen Starrkörper (Begründung siehe ADR-6).
- **Kontinuierliche Bewegungsglättung:** Um sprunghafte Posenänderungen zu vermeiden, führt der `ArticulationJointActuator` den Zielwert `xDrive.target` schrittweise pro Physikschritt (`FixedUpdate`) nach. Die Geschwindigkeit richtet sich nach dem übergebenen `speed`-Parameter. Ein neuer Befehl für dasselbe Gelenk bricht die laufende Interpolation ab (`Stop()`).
- **Horizontale Werkzeugausrichtung:** Reale Palettierroboter besitzen ein mechanisches Parallelogrammgestänge, welches das Werkzeug stets vertikal nach unten hält. In Unity wird dies im `LateUpdate`-Schritt über die Komponente `GripperDownConstraint` rechnerisch kompensiert (siehe ADR-7).

9.2. Szenen- und Objektlebenszyklus

Die Simulationsumgebung erlaubt das dynamische Erzeugen und Manipulieren von geometrischen Körpern (`Box`, `Sphere`, `Cylinder`).

- **Objekt-Instanziierung:** Python erzeugt Deklarationen (`WorldObject`) und sendet diese als JSON. Der `WorldSpawnManager` in Unity instanziiert das passende Prefab, weist Skalierung und Materialfarben zu und registriert das Objekt in der internen Verwaltung.
- **Physik- und Greifinteraktion:** Statische Objekte dienen als unbewegliche Hindernisse (`isKinematic = true`). Dynamische Objekte erhalten einen physikalischen `Rigidbody` sowie die Komponente `GrabbableObject`. Beim Greifen koppelt der Werkzeugcontroller das Objekt kinematisch an den Endeffektor (`AttachTo`); beim Loslassen (`Detach`) wird die physikalische Simulation reaktiviert.
- **Reproduzierbare Ausgangslagen:** Um deterministische Übungen zu ermöglichen, stellt `World.load_preset()` vordefinierte Szenarien bereit (Katalog siehe Kapitel 5, Entscheidung siehe ADR-5).

9.3. Schutz- und Validierungskonzept

Das Gesamtsystem schützt Roboter und Simulation durch ein zweistufiges Validierungsmodell:

- **Stufe 1 – Fail-Fast (Strukturprüfung):** Strukturelle Fehler (z. B. fehlender Port für den Hardwaremodus, leere Objektnamen, Vektoren mit falscher Dimensionierung) werden beim Methodenaufruf sofort abgefangen und lösen eine Python-Exception aus (z. B. `InvalidConfigError`).
- **Stufe 2 – Software-Clamping (Wertebereichsbegrenzung):** Zulässige numerische Parameter werden automatisch auf sichere Intervalle geklemmt, anstatt das Programm abzubrechen:
  - Gelenkwinkel werden auf die mechanischen Grenzen limitiert (Achsgrenzen siehe Kapitel 2). Unity sichert die Grenzen im `ArticulationBody` zusätzlich ab.
  - Geschwindigkeiten werden auf den Bereich `1..100` begrenzt.
  - RGB-Farbwerte werden auf den Wertebereich `0..255` geklemmt.

9.4. Test- und Qualitätssicherungskonzept

- **Python Unit Tests:** Automatisierte `pytest`-Suites validieren Protokollerstellung (`test_protocol.py`), Controller-Logik (`test_robot.py`) und Weltverwaltung (`test_world.py`) unter Einsatz von Socket- und Hardware-Mocks.
- **Unity EditMode Tests:** `NUnit`-Tests (`SerializationTests`, `ComponentTests`) stellen die korrekte JSON-Deserialisierung und das Vorhandensein erforderlicher Prefab-Komponenten sicher.
- **Manuelle Systemtests:** Das Zusammenspiel über das Netzwerk wird anhand strukturierter Testprotokolle manuell geprüft (automatisierte Integration siehe Kapitel 11).

10. Architecture Decisions

Die folgenden Architectural Decision Records (ADRs) dokumentieren die zentralen Richtungsentscheidungen, deren Begründung sowie die resultierenden Konsequenzen.

10.1. ADR-1: Einheitliche Python-API für Simulation und Hardware

- **Status:** Akzeptiert
- **Kontext:** Steuerungsskripte sollen ohne Codeänderungen zwischen Simulation und physischer Hardware übertragbar sein.
- **Entscheidung:** Die Klassen `Robot` und `World` stellen eine einheitliche Fassade bereit. Das Routing (`virtual`, `real`, `both`) erfolgt intern im `MyPalletizerController`.
- **Betrachtete Alternativen:**
  1. Getrennte Bibliotheken für Simulation und Hardware.
  2. Direkte Weitergabe der Herstellerbibliothek `pymycobot`.
- **Begründung:** Eine gemeinsame Schnittstelle minimiert den Lernaufwand und ermöglicht die direkte Wiederverwendung von Beispielcode.
- **Konsequenzen:**
- **Positiv:** Identische Syntax für alle Betriebsmodi; Entkopplung des Benutzercodes von Netzwerk- und Protokolldetails.
- **Negativ:** Zusätzliche Abstraktionsschicht zur Verwaltung von Moduslogik und Validierung.

10.2. ADR-2: Kapselung der Herstellerbibliothek `pymycobot`

- **Status:** Akzeptiert
- **Kontext:** Die herstellereigene Bibliothek `pymycobot` steuert den physischen Roboter, unterliegt jedoch herstellerseitigen API-Änderungen.
- **Entscheidung:** `pymycobot` wird ausschliesslich intern im `MyPalletizerController` gekapselt.
- **Betrachtete Alternativen:**
  1. Direkte Verwendung von `pymycobot` durch den Anwender.
  2. Komplette Neuentwicklung eines eigenen seriellen Treibers.
- **Begründung:** Die Kapselung schützt die öffentliche API vor herstellerspezifischen Schnittstellenbrüchen und ermöglicht zentrales Software-Clamping.
- **Konsequenzen:**
- **Positiv:** Stabile, typisierte Schnittstelle; zentrale Fehlerbehandlung.
- **Negativ:** Neue herstellerspezifische Funktionen müssen manuell in die Fassade integriert werden.

10.3. ADR-3: Hybride Betriebsmodi (`virtual`, `real`, `both`)

- **Status:** Akzeptiert
- **Kontext:** Es wird sowohl isoliertes Offline-Testing als auch der direkte Vergleich zwischen 3D-

Modell und physischer Bewegung benötigt.

- **Entscheidung:** Unterstützung von drei festen Betriebsarten: `virtual` (nur Unity), `real` (nur Hardware) und `both` (Parallelbetrieb).
- **Betrachtete Alternativen:**
  1. Reine Simulationslösung.
  2. Umschalten über externe Konfigurationsdateien statt nativer Klassenparameter.
- **Begründung:** Der Modus `both` erlaubt den synchronen visuellen Abgleich von Kinematikmodell und Servo-Verhalten im Labor.
- **Konsequenzen:**
- **Positiv:** Hohe Flexibilität für Studierende im Selbststudium und im Labor.
- **Negativ:** Im Modus `both` können minimale zeitliche Abweichungen zwischen Simulation und Hardware entstehen.

10.4. ADR-4: Trennung der Kommunikationskanäle (UDP Ports 5005 und 5006)

- **Status:** Akzeptiert
- **Kontext:** Das System überträgt zeitkritische Gelenkbefehle sowie datenintensivere Szenen- und Objektbefehle.
- **Entscheidung:** Roboterkommandos werden über UDP-Port `5005` gesendet, Weltbefehle über UDP-Port `5006`.
- **Betrachtete Alternativen:**
  1. Gemeinsamer UDP-Kanal für alle Nachrichtentypen.
  2. Einsatz von TCP oder WebSockets.
- **Begründung:** Jeder Kanal hat in Unity eigenen Empfangs-Thread und eigene Warteschlange. Roboterbefehle und Weltbefehle stören sich dadurch nicht gegenseitig, zum Beispiel beim Laden eines Presets mit vielen Objekten.
- **Konsequenzen:**
- **Positiv:** Klare funktionale Entkopplung; Weltbefehle warten nicht auf Roboterbewegungen. Innerhalb des Roboterkanals gibt es aber weiterhin nur eine Queue: Ein `sync_move_joints` hält die folgenden Roboterbefehle an.
- **Negativ:** Zwei lokale Netzwerkports müssen freigehalten und konfiguriert werden.

10.5. ADR-5: Bereitstellung vordefinierter Szenarien über Presets

- **Status:** Akzeptiert
- **Kontext:** In Übungen müssen reproduzierbare Ausgangssituationen für Pick-and-Place-Aufgaben schnell geladen werden können.

- **Entscheidung:** Die vorhandenen Standardszenarien (`empty`, `three_blocks`, `Base1`, `sorting_task`) werden als Python-Presets definiert und über `World.load_preset()` bereitgestellt.
- **Betrachtete Alternativen:**
  1. Manuelles Platzieren aller Objekte im Unity-Editor vor Programmstart.
  2. Laden ausschliesslich aus externen JSON-Dateien.
- **Begründung:** Codebasierte Presets ermöglichen deterministische und automatisierbare Übungsabläufe direkt aus dem Steuerungsskript. Ein eigenes `pick_and_place_basic`-Preset ist derzeit nicht implementiert.
- **Konsequenzen:**
- **Positiv:** Sofort einsatzbereite, reproduzierbare Laboraufgaben.
- **Negativ:** Presets müssen bei Erweiterung der 3D-Körper-Bibliothek nachgepflegt werden.

10.6. ADR-6: Einsatz von `ArticulationBody` für die Gelenkphysik in Unity

- **Status:** Akzeptiert
- **Kontext:** Der Roboterarm besteht aus vier rotatorisch verbundenen Starrkörpern, die Lasten aufnehmen und stabil bewegt werden müssen.
- **Entscheidung:** Verwendung von `ArticulationBody`-Komponenten statt herkömmlicher `Rigidbody` - und `HingeJoint`-Ketten.
- **Betrachtete Alternativen:**
  1. Rein visuelle Rotation über `Transform`-Hierarchien ohne Physikberechnung.
  2. Physiksimulation über Standard-`Rigidbody` mit Gelenken.
- **Begründung:** `ArticulationBody` wurde speziell für Robotiksimulationen entwickelt und verhindert numerische Gelenkdehnungen zuverlässig.
- **Konsequenzen:**
- **Positiv:** Stabiles Mehrkörperverhalten; exakte Gelenkbeschränkungen.
- **Negativ:** Höhere Konfigurationskomplexität im Unity-Inspector.

10.7. ADR-7: Pragmatische Nutzung herstellerseitiger CAD-Modelle mit Kompensation

- **Status:** Akzeptiert
- **Kontext:** Die bereitgestellten 3D-CAD-Modelle des Herstellers weisen minimale geometrische Toleranzen zur Hardware auf.
- **Entscheidung:** Verwendung der Hersteller-CAD-Modelle mit softwareseitiger Ausrichtungskompensation (`GripperDownConstraint`) in Unity.
- **Betrachtete Alternativen:**

1. Vollständige Neukonstruktion aller 3D-Modelle im CAD-System.
2. Verwendung stark vereinfachter Platzhalterkörper.

- **Begründung:** Eine Neukonstruktion stand ausserhalb des Projektzeitrahmens. Die softwareseitige Ausrichtung liefert eine für Ausbildungszwecke vollkommen ausreichende visuelle und funktionale Genauigkeit.
- **Konsequenzen:**
- **Positiv:** Realistischer optischer Eindruck bei minimalem Modellierungsaufwand.
- **Negativ:** Für mikrometergenaue Kollisionsprüfungen ist das Modell nicht geeignet.

10.8. ADR-8: Verzicht auf optische Kamerasimulation zugunsten der Kernkinematik

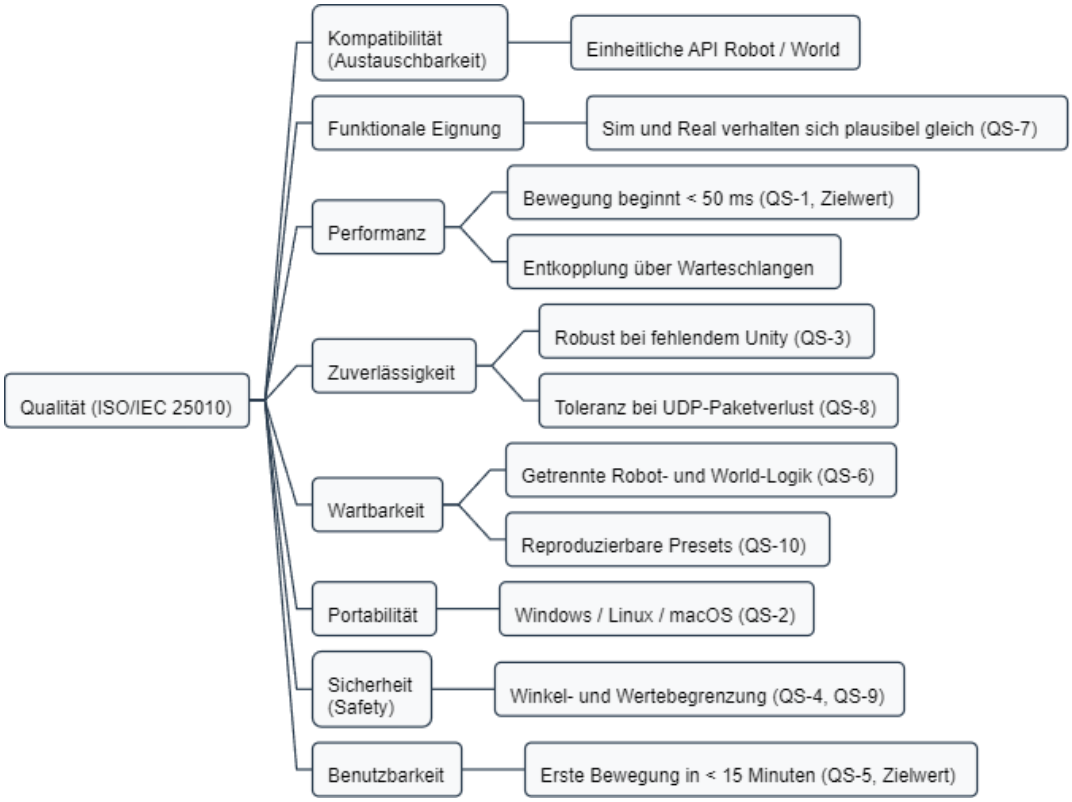
- **Status:** Akzeptiert
- **Kontext:** Der reale MyPalletizer kann mit einer USB-Kamera betrieben werden. Kamerasimulationen unterliegen jedoch komplexen Umweltfaktoren (Beleuchtung, Rauschen).
- **Entscheidung:** Das System fokussiert auf Kinematik, Werkzeuge und Objektmanipulation; auf eine Kamerasimulation in Unity wird verzichtet.
- **Betrachtete Alternativen:**
  1. Virtuelle Unity-Kamera mit Bildstream an OpenCV.
- **Begründung:** Das Hauptziel ist die Programmierung von Manipulations- und Sortieraufgaben. Bildverarbeitung wird an der HSLU in einem anderen Modul unterrichtet.
- **Konsequenzen:**
- **Positiv:** Schlanke Architektur, geringe Renderlast, fokussierte Codebasis.
- **Negativ:** Bildverarbeitungs-Pipelines können nicht virtuell vortrainiert werden.

11. Quality Requirements

Die Qualitätsanforderungen konkretisieren die Qualitätsziele aus Kapitel 1 anhand eines Qualitätsbaums sowie messbarer Qualitätsszenarien.

11.1. Quality Tree

Der Qualitätsbaum ordnet die Anforderungen den ISO/IEC 25010 Qualitätsmerkmalen zu:



11.2. Quality Scenarios

ID	Merkmal	Szenario (Stimulus & Kontext)	Messbares Akzeptanzkriterium
QS-1	**Performanz**	Ein Benutzer sendet Gelenkbewegungen (`send_angles`) über die Python-API an die laufende Unity-Simulation.	Zielwert: Die visuelle Bewegung des Roboterarms soll innerhalb von `50 ms` beginnen. Ein Messprotokoll ist im Repository nicht enthalten.
QS-2	**Portabilität**	Ein Steuerungsskript wird unter Windows entwickelt und anschliessend auf macOS oder Linux ausgeführt.	Das Skript läuft ohne Codeanpassungen fehlerfrei aus.
QS-3	**Robustheit**	Die Unity-Simulation wird während der Programmausführung unerwartet beendet oder ist noch nicht gestartet.	Zielwert: Das Python-Skript soll bei nicht erreichbarem Unity-Prozess nicht auf eine Antwort warten. Ein Netzwerkfehler kann derzeit trotzdem als Socket-Exception auftreten.
QS-4	**Eingabeschutz**	Ein Benutzer übergibt einen Gelenkwinkel von `200°` für Joint 1 (zulässig: `-160°..160°`).	Der Controller fängt den Wert ab und klemmt ihn automatisch auf `160°` (Software-Clamping).

ID	Merkmal	Szenario (Stimulus & Kontext)	Messbares Akzeptanzkriterium
QS-5	**Benutzbarkeit**	Ein neuer Anwender ohne Unity-Vorkenntnisse installiert das Paket und führt ein Beispielskript aus.	Zielwert: Ein neuer Anwender soll die erste Roboterbewegung in unter `15 Minuten` ausführen können. Dieser Wert ist bisher nicht gemessen.
QS-6	**Wartbarkeit**	Ein Entwickler ergänzt einen neuen administrativen Weltbefehl in Python und C#.	Die Änderung soll auf Protokollmodelle, UDP-Bridge, `WorldSpawnManager` und die zugehörigen Tests begrenzt bleiben; die Roboterklassen sollen unverändert bleiben.
QS-7	**Funktionale Korrektheit**	Ein Pick-and-Place-Ablauf wird synchron im Modus `both` ausgeführt.	Zielwert: Gelenkrichtungen und Werkzeugaktivierungen sollen visuell plausibel mit der Hardware übereinstimmen. Es gibt keinen automatisierten Vergleich.
QS-8	**Fehlertoleranz**	Während einer Bewegung geht ein einzelnes UDP-Datagramm im lokalen Netzwerk verloren.	Für Bewegungen kann ein späteres Paket den Zielzustand korrigieren. Einzelne Befehle werden nicht quittiert oder wiederholt.
QS-9	**Sicherheit**	Ein Benutzer übergibt Geschwindigkeitswerte von `250` oder negative RGB-Werte.	Geschwindigkeiten werden auf `1..100` und RGB-Werte auf `0..255` geklemmt.
QS-10	**Reproduzierbarkeit**	Ein Praktikumsszenario soll deterministisch zurückgesetzt werden.	Der Aufruf `World.load_preset("three_blocks")` reinigt die Szene und instanziiert die Objekte an exakt definierten Startkoordinaten.

11.3. Architektonische Zielkonflikte (Trade-offs)

- **Latenz vs. Zustellung:** Verbindungsloses UDP priorisiert minimale Latenz. Bewegungen können durch spätere absolute Zielwerte korrigiert werden; eine garantierte Zustellung gibt es nicht.

- **Einfachheit der API vs. Hardware-Transparenz:** Die High-Level-Fassade verbirgt feingranulare herstellerspezifische Sonderfunktionen von `pymycobot`, um eine einheitliche und intuitive Schnittstelle zu bieten.
- **Visuelle Plausibilität vs. Hochpräzise Dynamik:** Die Simulation priorisiert eine performante Echtzeitdarstellung in Unity gegenüber einer rechenintensiven Finite-Elemente-Simulation.

12. Risks and Technical Debts

Dieses Kapitel dokumentiert bekannte architektonische und technologische Risiken sowie bestehende technische Schulden und Code-Einschränkungen mit Massnahmen zu deren Beherrschung.

12.1. Architektur- und Technologierisiken

Risiko	Ursache und Auswirkung	Massnahme zur Risikominderung
Paketverlust bei UDP-Kommunikation	UDP garantiert weder Zustellung noch Reihenfolge. Einzelne verlorene Pakete könnten zu Fehlstellungen führen.	Idempotentes Protokolldesign: Jedes Bewegungsdatagramm enthält den vollständigen Zustand (`j1` bis `j4`). Nachfolgende Befehle korrigieren Posenabweichungen automatisch.
Laufzeit-Asynchronität im Modus `both`	Unity-Physik und reale Servomotoren weisen unterschiedliche Ansprechzeiten und Geschwindigkeiten auf.	Beide Zielsysteme erhalten identische Sollwinkel. Python synchronisiert bei `sync_move_joints` auf die Hardware, während Unity den Befehl parallel in seiner Queue verarbeitet.
Thread-Sicherheit in Unity	Fehlerhafte Zugriffe aus dem Netzwerkempfangs-Thread auf Unity-Objekte führen zu Abstürzen der Engine.	Strikte Entkopplung über threadsichere Queues mit Sperren (`lock`). Die Befehlsabarbeitung erfolgt ausschliesslich im Unity-Hauptthread (`CommandRunner`).
Abhängigkeit von `pymycobot`	Änderungen oder Fehler in der Herstellerbibliothek könnten die Hardwareansteuerung beeinträchtigen.	Kapselung von `pymycobot` im `MyPalletizerController`. Bei Schnittstellenänderungen muss nur dieser Adapter angepasst werden.

12.2. Bekannte Einschränkungen und technische Schulden

Die folgende Übersicht fasst die identifizierten Einschränkungen im Quellcode und die dokumentierten technischen Schulden zusammen.

Einschränkung / Technische Schuld	Auswirkung in der Praxis	Massnahme / Geplante Weiterentwicklung
Open-Loop-Simulation (Fehlender Rückkanal)	Python erhält keine Quittung und keine simulierten Ist-Werte von Unity. `get_angles()` liest den lokalen Speicher `_sim_angles`.	Für Ausbildungszwecke ausreichend; bei Fehlern bietet das Unity-Konsolenlog Diagnosehinweise. Langfristig ist ein Telemetrie-Rückkanal (z. B. auf Port `5007`) vorbereitet.
Werkzeugauswahl als Vorbedingung	Werkzeugbefehle (`pump_on`, `set_gripper_state`) werden von Unity ignoriert, wenn das jeweilige Werkzeug nicht zuvor per `set_end_effector` aktiviert wurde.	In Steuerungsskripten vor Werkzeugaktionen stets `set_end_effector` aufrufen. Eine Validierungsmeldung in der Python-API ist empfohlen.
Getrennte manuelle Port-Konfiguration	Ports `5005` und `5006` sind in Python per Parameter anpassbar, in Unity jedoch nur im Editor-Inspector fest verdrahtet.	Ports bei Änderungen auf beiden Seiten synchron einstellen. Mittelfristig eine gemeinsame Konfigurationsdatei (z. B. JSON/YAML) anbinden.
Vereinfachtes Greifermodell	Werkzeuge nutzen kinematische Fixierung (`AttachTo`) anstelle physikalischer Reibungskraftberechnung.	Für Lehrzwecke stabil und deterministisch. Ein physikalisches Reibungsmodell kann bei Bedarf als modularer Werkzeugcontroller nachgerüstet werden.
Fehlende automatisierte End-to-End-Tests	Unit-Tests (Python) und EditMode-Tests (Unity) laufen automatisiert; die Netzwerk-Gesamtkette wird bisher manuell nach Protokoll getestet.	Einrichtung eines Headless-Unity-Testrunners in der GitHub-Actions-CI-Pipeline.

